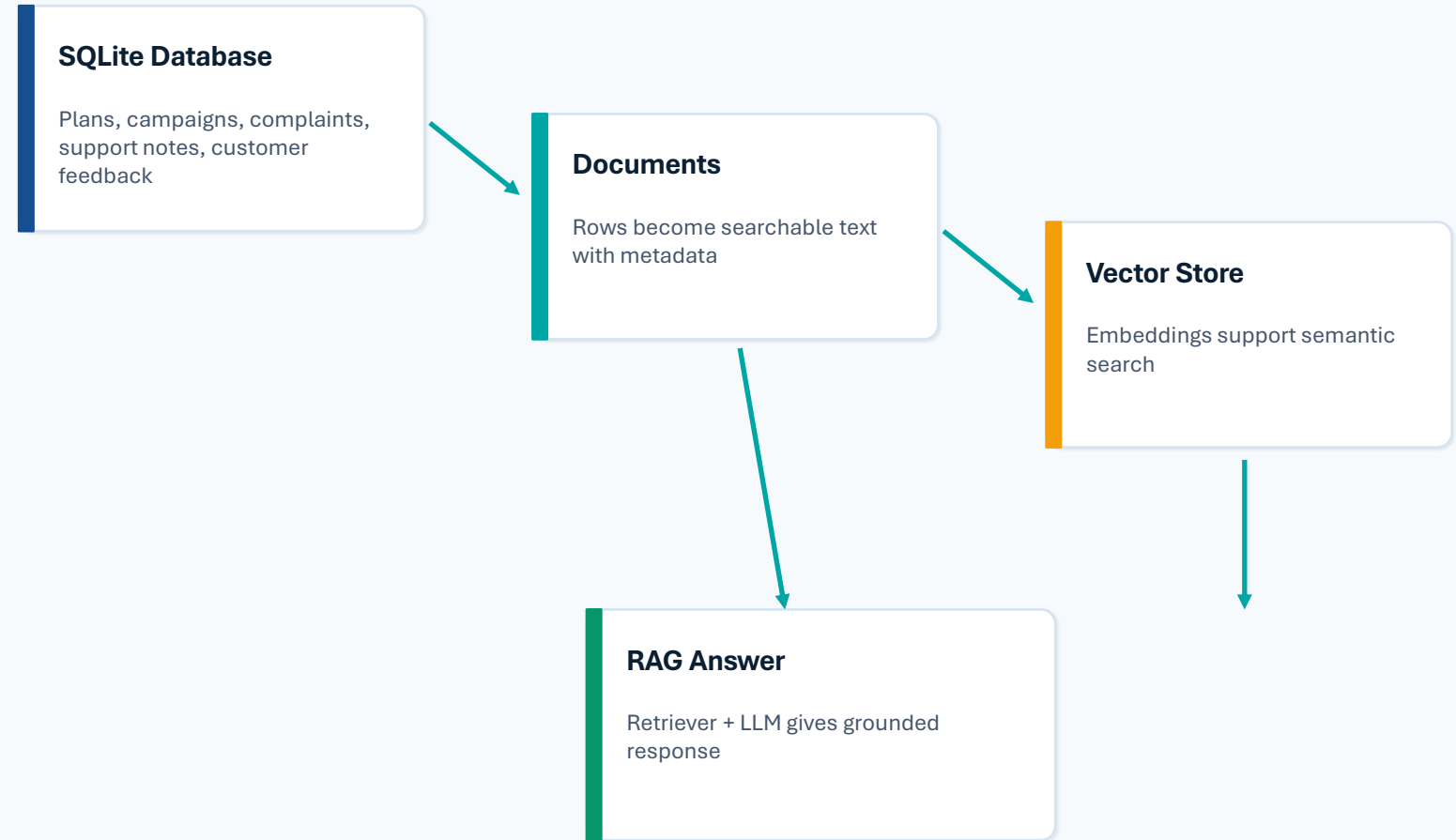


RAG Assistant from Telecom Database Content

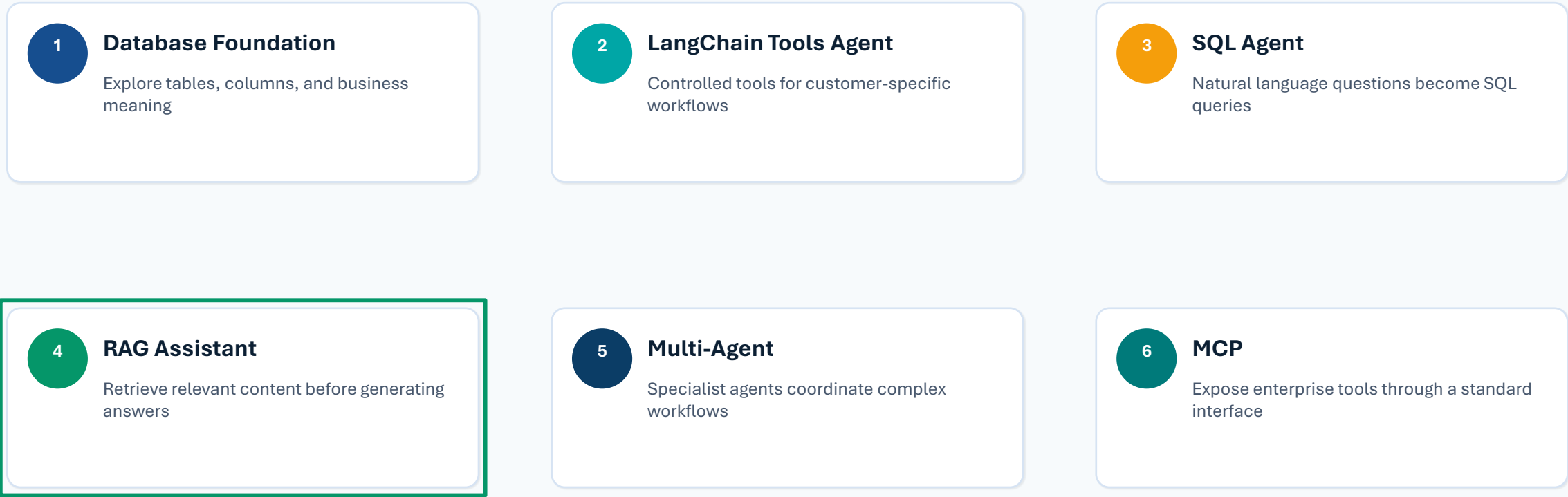
Zain Jordan Customer 360 AI
Workshop



Class outcome: build a retrieval assistant that recommends plans, offers, and support guidance using selected database content.

Where Workshop 4 Fits

The learning path now moves from database querying to retrieval-based AI answers.



Workshop 4 teaches when SQL is not enough: we need semantic retrieval over plans, offers, policies, complaints, and support notes.

From SQL Agent to RAG Assistant

SQL Agent

Best for structured questions:

- counts and totals
- grouping and ranking
- joins across tables
- numeric business analysis

RAG Assistant

Best for meaning-based questions:

- plan suitability
- campaign matching
- complaint themes
- support guidance
- explanation and recommendation

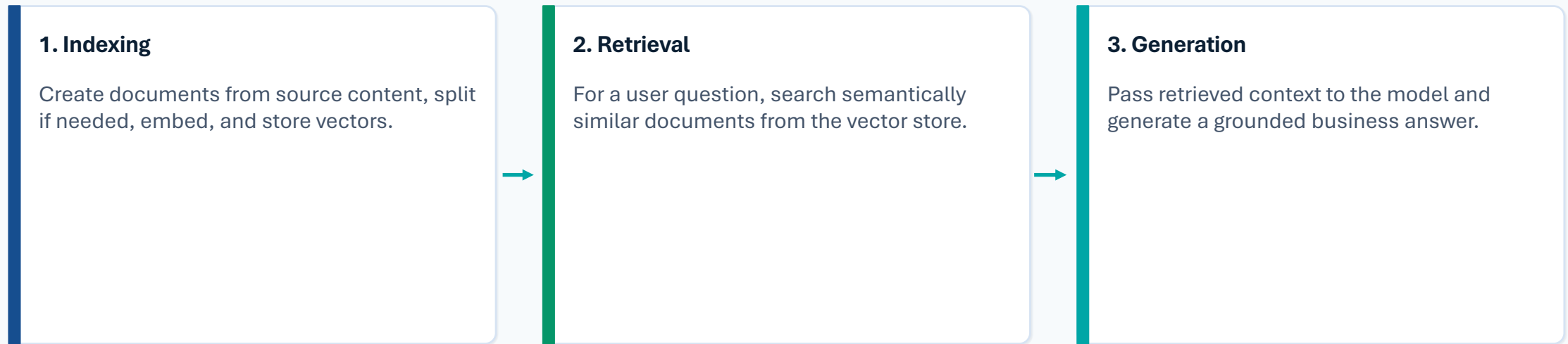
Hybrid System

Best for real solutions:

- SQL finds the customer facts
- RAG retrieves relevant knowledge
- LLM writes a grounded answer

Key message: RAG does not replace SQL. It complements SQL when answers require meaning, context, and explanation.

What RAG Means



In this class, source content comes from selected rows in the same Zain Jordan SQLite database.

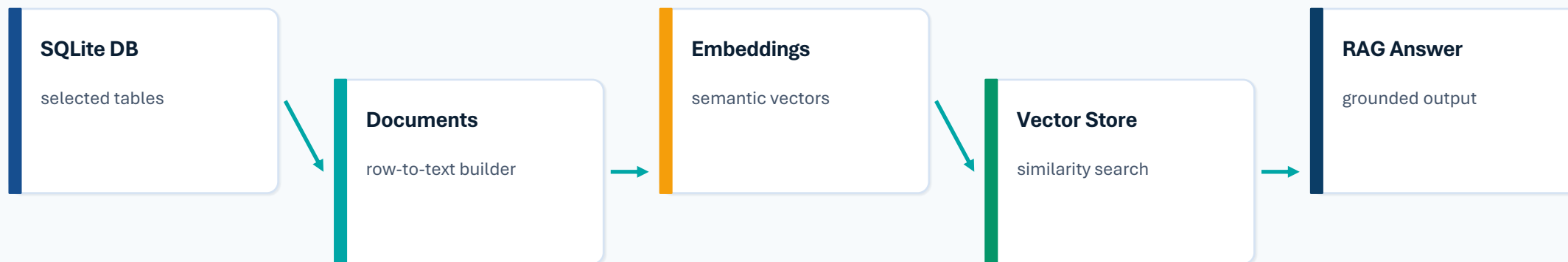
Which Database Content Becomes RAG?

Not every table should become a vector document. Choose content that benefits from semantic search.

Table	Text to retrieve	Training use case
plans	Plan names, fees, data, minutes, contract terms	Plan recommendation assistant
addons	Extra data, roaming, voice, SMS bundles	Add-on recommendation
campaigns	Offer descriptions and target segments	Campaign matching
complaints	Complaint category + descriptions + resolution summaries	Complaint theme search
support_interactions	Issues, channels, sentiment, resolution notes	Support guidance
customer_satisfaction	NPS, CSAT, feedback text, sentiment	Customer experience insights

Rule: SQL is for exact calculations. RAG is for matching meaning and producing contextual explanations.

Workshop 4 Architecture

**Example RAG Flow**

Question: "Which offer fits a VIP customer with roaming usage?"

Retriever finds:

- roaming add-ons
- campaigns for high-value customers
- support notes about roaming billing

LLM writes:

- recommendation
- explanation
- customer message

Row-to-Document Design

A database row must be converted into a clear text document before embedding.

Document text example

Plan Name: Family 50
Category: Consumer Mobile
Monthly Fee: 50 JOD
Data Allowance: 120 GB
Local Minutes: Unlimited
Technology: 5G
Contract: 12 months
Best For: high-data family usage

Metadata to keep

- table: plans
- record_id: plan_id
- category: Consumer Mobile
- service_type: Mobile
- source: Zain Customer 360 DB

Why metadata matters

It allows filtering, auditability, source display, and better debugging when the answer is wrong.

Good RAG starts before embeddings: write clear documents with traceable metadata.

Class Build Plan

1

Select tables

Choose content-rich tables: plans, addons, campaigns, complaints, support notes

2

Create documents

Convert selected rows into readable text + metadata

3

Embed documents

Use embedding model to convert text into vectors

4

Store vectors

Use an in-memory vector store for beginner simplicity

5

Retrieve context

Search relevant documents for a user question

6

Generate answer

Pass retrieved context to LLM for grounded response

Demo 1 — Build Documents from Database Rows

The first demo converts structured telecom rows into text documents that retrieval can understand.

Core pattern

```
for row in plans:
    text = f"""
    Plan: {row.plan_name}
    Category: {row.plan_category}
    Monthly Fee: {row.monthly_fee_jod} JOD
    Data: {row.data_allowance_gb} GB
    Minutes: {row.local_minutes}
    """
    documents.append(Document(
        page_content=text,
        metadata={"table": "plans", "id": row.plan_id}
    ))
```

What learners should notice

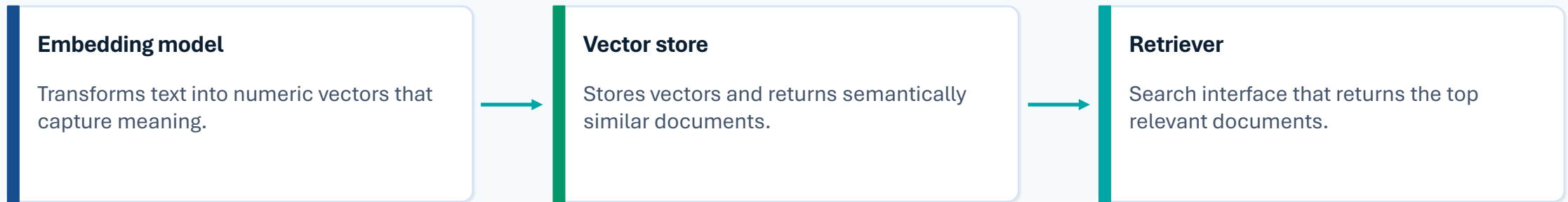
- Documents are not raw rows
- Text should be business-readable
- Metadata preserves traceability
- Tables become searchable knowledge

Beginner-friendly scope

Start with 3–4 tables only. Do not vectorize the full database in the first demo.

Demo 2 — Create Embeddings and Vector Store

Embeddings make semantic search possible: the assistant can find related content even when words do not match exactly.



Notebook concept

```
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = InMemoryVectorStore(embeddings)
vector_store.add_documents(documents)
retriever = vector_store.as_retriever(search_kwargs={"k": 4})
```

Demo 3 — Retrieval Before Generation

Before asking the LLM to answer, we first inspect what documents retrieval returns.

Retrieval test

```
query = "best offer for a heavy data user with roaming needs"  
results = retriever.invoke(query)
```

```
for doc in results:  
    print(doc.metadata)  
    print(doc.page_content[:500])
```

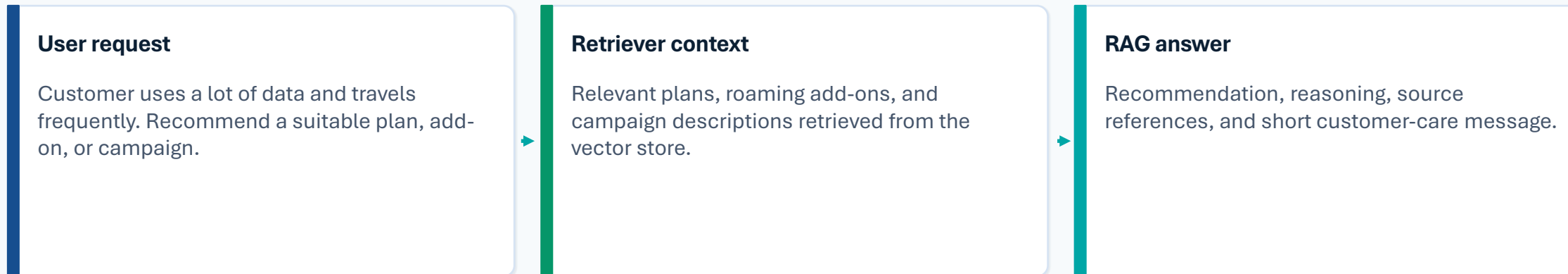
Why test retrieval?

If retrieved documents are poor, the final answer will be poor. RAG quality begins with retrieval quality.

Debug checklist

- Are documents relevant?
- Are top results diverse?
- Is metadata visible?
- Is k too low or too high?
- Are documents too short or too long?

Demo 4 — Plan & Offer RAG Assistant



Output should always say what evidence it used. This is the difference between a useful assistant and a fluent guesser.

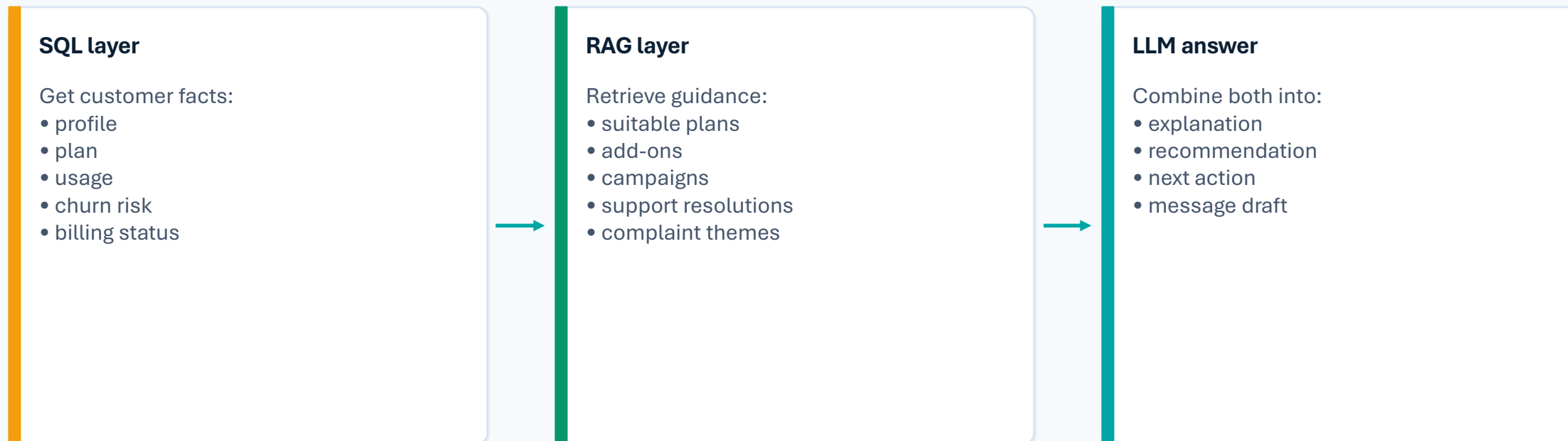
Structured response

Final answer format:

1. Recommended plan / add-on / campaign
2. Why it fits the customer need
3. Evidence from retrieved documents
4. Confidence / limitations
5. Customer care message

Hybrid Pattern — SQL + RAG

The best capstone projects can combine exact customer facts from SQL with semantic guidance from RAG.



Workshop 4 builds the RAG side. Later workshops connect it with agents and MCP.

Grounded Answer Checklist

Relevant retrieval

Top documents match the user question

Source visibility

Answer can point back to table and record

No unsupported claims

No recommendation without evidence

Clear limitations

If data is missing, say so

Business action

Answer ends with next step, not just summary

Participant Exercises

Exercise 1: Retrieval test

Query the vector store for:
“best add-on for roaming customer”
Review the top 4 documents.

Exercise 2: Plan advisor

Ask the RAG assistant to recommend a plan for a heavy data user.

Exercise 3: Complaint theme

Retrieve support/complaint records for “billing confusion” and summarize patterns.

Exercise 4: Improve documents

Add more fields to document text. Compare retrieval quality before and after.

Exercise 5: Source-aware answer

Modify the final answer to mention which table/record supported the response.

Capstone Ideas Using RAG

Plan Recommendation Assistant

Recommends plans/add-ons using customer need and plan knowledge

Campaign Match Assistant

Matches customer segment to relevant campaigns and offers

Support Resolution Assistant

Finds similar complaint/support cases and suggests resolution path

Customer Message Assistant

Generates grounded care messages using retrieved policies/offers

Knowledge Search Assistant

Semantic search over telecom plans, add-ons, campaigns, and feedback

Hybrid Customer 360 Advisor

Combines SQL customer facts with RAG recommendations

Common RAG Mistakes

Vectorizing everything

The full database is not automatically useful. Select tables that contain meaning-rich text.

Weak document text

If rows are converted poorly, retrieval quality will suffer even with a good model.

No source tracking

Without metadata, users cannot verify why the assistant gave an answer.

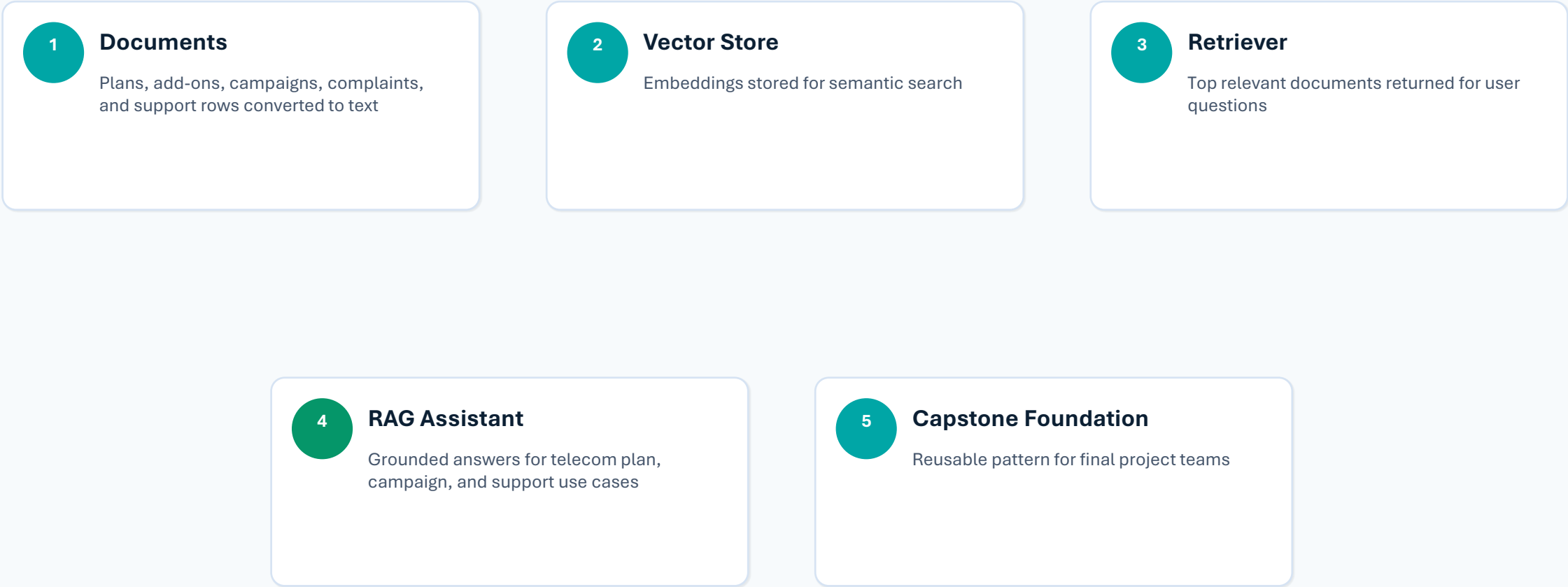
Answering beyond context

The model should not invent policies, prices, or offers not present in retrieved context.

Skipping evaluation

Always test retrieval quality, answer grounding, and business usefulness.

What Learners Build Today



Transition to the Next Workshops

Next: Multi-Agent

Specialist agents can use SQL and RAG as tools. Example: Customer Agent + Offer Agent + Support Agent.

Then: MCP

The same database and RAG functions can be exposed as standardized enterprise tools.

Final: Capstone

Teams build a practical telecom AI assistant using the shared Zain Jordan database.

The pattern is now complete: structured SQL insights + semantic RAG recommendations + agentic orchestration.

Reference Concepts

Workshop 4 uses standard retrieval-augmented generation concepts from the LangChain ecosystem.

Core RAG flow

Index source data, retrieve relevant context at question time, then generate an answer from the retrieved context.

Retrieval building blocks

Document loaders, text splitters, embeddings, vector stores, and retrievers.

Workshop adaptation

Instead of external documents, we create documents from selected telecom database rows.

Source note: Official LangChain RAG, retrieval, embeddings, and vector store documentation were used to align the workshop language.